

# Smart Crop-Ai Powered Crop Recommendation Sysytem

## Project Description:

OptiCrop is a machine learning-based web application that recommends the most suitable crop for cultivation based on soil and environmental parameters. The application helps farmers and agricultural professionals make informed decisions using Artificial Intelligence.

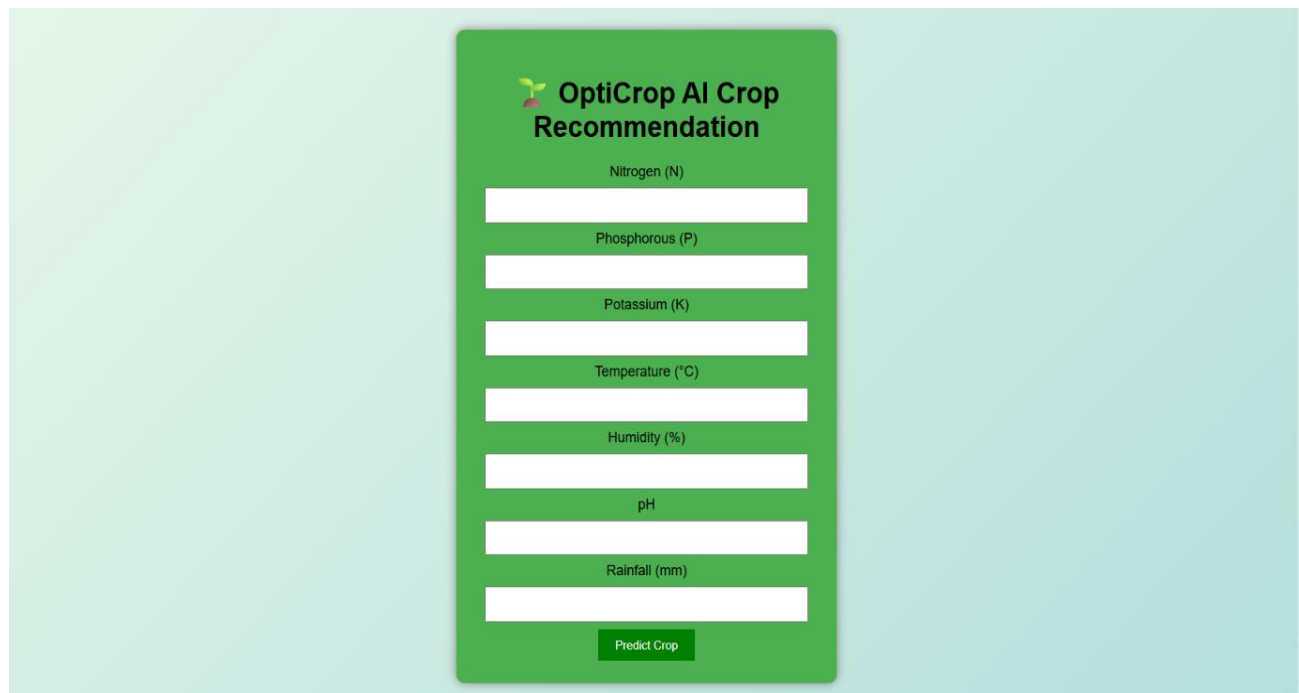
The system takes the following inputs:

- Nitrogen
- Phosphorus
- Potassium
- Temperature
- Humidity
- pH Value
- Rainfall

After receiving these values, the trained Machine Learning model predicts the most appropriate crop.

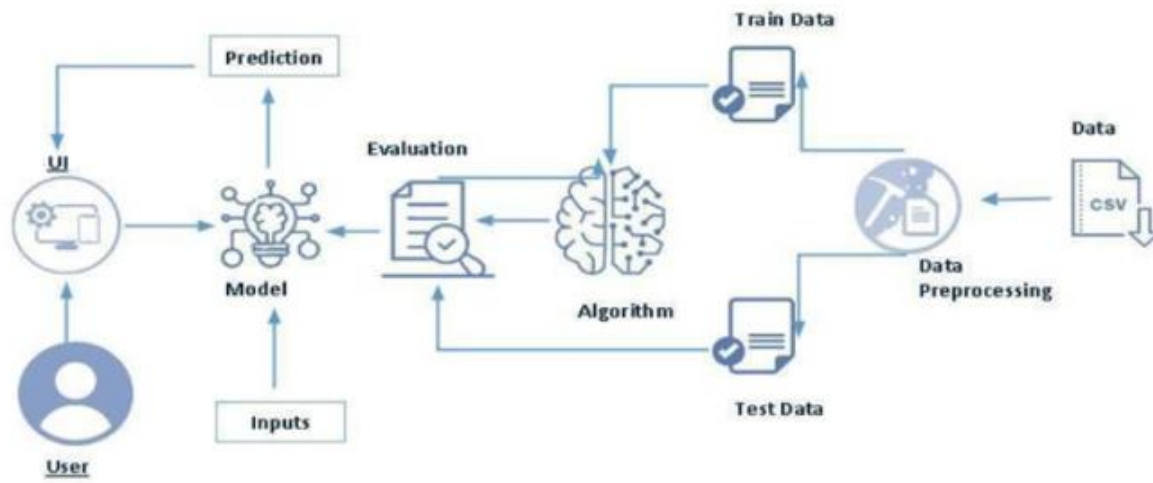
The application is developed using Python, Flask, HTML, CSS, JavaScript, and Scikit-learn. The trained model is saved using Pickle and integrated into the Flask application. The project is hosted on Render for public access.

## Project Interface:



The screenshot displays the OptiCrop AI Crop Recommendation web application interface. The interface is a green card with a white border, set against a light blue background. At the top, there is a small green plant icon followed by the title "OptiCrop AI Crop Recommendation". Below the title, there are eight input fields, each with a label above it: "Nitrogen (N)", "Phosphorous (P)", "Potassium (K)", "Temperature (°C)", "Humidity (%)", "pH", and "Rainfall (mm)". Each input field is a white rectangular box. At the bottom of the card, there is a green button with the text "Predict Crop" in white.

## Technical Architecture:



## Description:

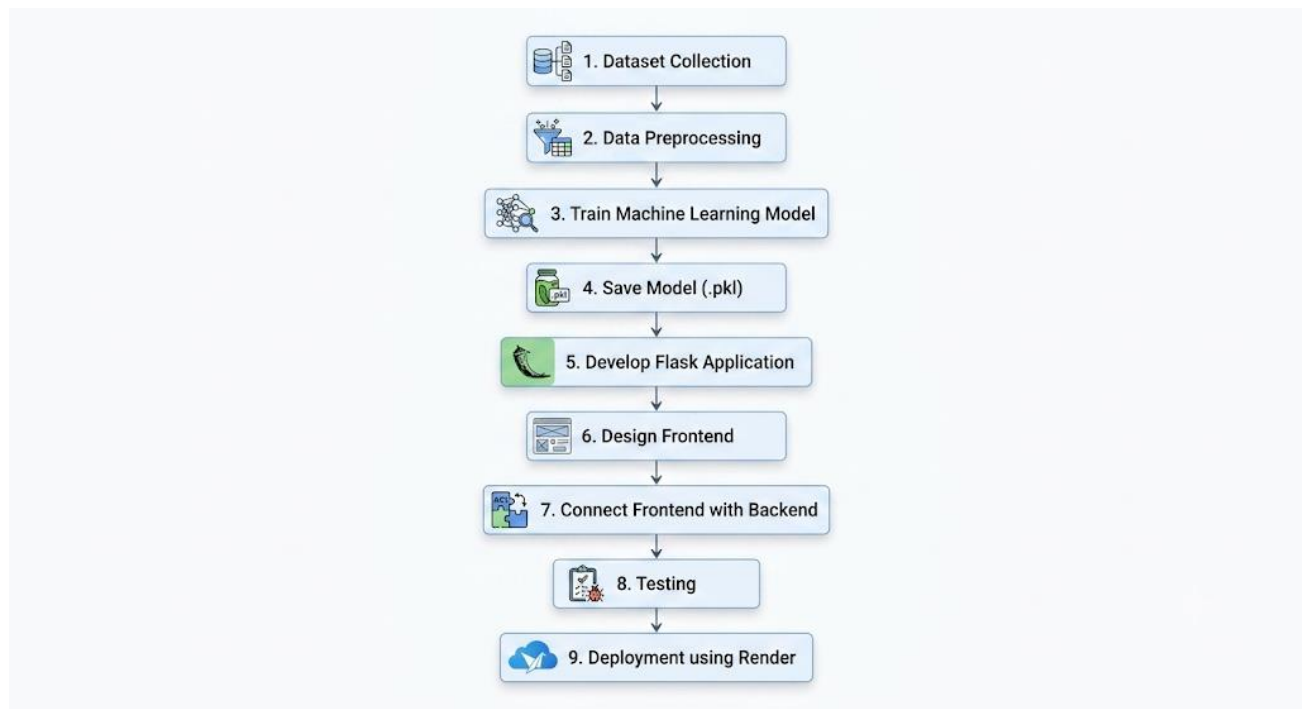
This diagram maps out a standard Machine Learning Workflow from start to finish:

1. **Data Prep:** Raw data (CSV) is preprocessed and split into Train Data (to build the model) and Test Data (to check its accuracy).
2. **Training & Evaluation:** An Algorithm learns from the training data, and the resulting system is Evaluated against the test data.
3. **Deployment & Prediction:** The finalized Model takes new inputs from a User via a UI and delivers a Prediction back to them.

## Pre-requisites:

- **Flask Framework Knowledge:** [Flask Documentation](#)
- **Groq API Familiarity:** [Groq API Documentation](#)
- **HTML, CSS, and JavaScript Skills:** [W3Schools HTML/CSS/JavaScript Tutorials](#)
- **Python Programming Proficiency:** [Python Documentation](#)
- **Version Control with Git:** [Git Documentation](#)
- **Development Environment Setup:** [Flask Installation Guide](#)

## Project Workflow:



## Milestone 1: Dataset Collection

The Crop Recommendation dataset was collected from a reliable agricultural dataset source. It contains soil nutrient values, environmental parameters, and corresponding crop labels used for training the machine learning model. Features:

- Nitrogen
- Phosphorus
- Potassium
- Temperature
- Humidity
- pH
- Rainfall

**Target:** Crop Name

**Dataset :**

|    | A  | B  | C  | D          | E          | F          | G          | H     |  |
|----|----|----|----|------------|------------|------------|------------|-------|--|
| 1  | N  | P  | K  | temperatur | humidity   | ph         | rainfall   | label |  |
| 2  | 90 | 42 | 43 | 20.879744  | 82.002744  | 6.50298529 | 202.93554  | rice  |  |
| 3  | 85 | 58 | 41 | 21.770462  | 80.319644  | 7.0380964  | 226.65554  | rice  |  |
| 4  | 60 | 55 | 44 | 23.004459  | 82.320763  | 7.8402071  | 263.96425  | rice  |  |
| 5  | 74 | 35 | 40 | 26.491096  | 80.158363  | 6.9804009  | 242.86403  | rice  |  |
| 6  | 78 | 42 | 42 | 20.130175  | 81.604873  | 7.6284729  | 262.71734  | rice  |  |
| 7  | 69 | 37 | 42 | 23.058049  | 83.370118  | 7.0734535  | 251.055    | rice  |  |
| 8  | 69 | 55 | 38 | 22.708838  | 82.639414  | 5.7008057  | 271.32486  | rice  |  |
| 9  | 94 | 53 | 40 | 20.277744  | 82.894086  | 5.71862717 | 241.97419  | rice  |  |
| 10 | 89 | 54 | 38 | 24.515881  | 83.5352162 | 6.6853464  | 230.44624  | rice  |  |
| 11 | 68 | 58 | 38 | 23.223974  | 83.033227  | 6.3362535  | 221.2092   | rice  |  |
| 12 | 91 | 53 | 40 | 26.527235  | 81.417538  | 5.3861678  | 264.61487  | rice  |  |
| 13 | 90 | 46 | 42 | 23.978982  | 81.450616  | 7.502834   | 250.08323  | rice  |  |
| 14 | 78 | 58 | 44 | 26.800796  | 80.886848  | 5.1086818  | 284.43646  | rice  |  |
| 15 | 93 | 56 | 36 | 24.014976  | 82.056872  | 6.9843537  | 185.27734  | rice  |  |
| 16 | 94 | 50 | 37 | 25.665852  | 80.66385   | 6.9480198  | 209.58697  | rice  |  |
| 17 | 60 | 48 | 39 | 24.282094  | 80.300256  | 7.04229906 | 231.08633  | rice  |  |
| 18 | 85 | 38 | 41 | 21.587118  | 82.788371  | 6.24905065 | 276.655245 | rice  |  |
| 19 | 91 | 35 | 39 | 23.79392   | 80.41818   | 6.9708598  | 206.26119  | rice  |  |
| 20 | 77 | 38 | 36 | 21.865252  | 80.192301  | 5.9539333  | 224.555016 | rice  |  |
| 21 | 88 | 35 | 40 | 23.579436  | 83.587603  | 5.8539321  | 291.298661 | rice  |  |
| 22 | 89 | 45 | 36 | 21.325042  | 80.474764  | 6.4424754  | 185.49747  | rice  |  |
| 23 | 76 | 40 | 43 | 25.157455  | 83.117135  | 5.0701757  | 231.38432  | rice  |  |
| 24 | 67 | 59 | 41 | 21.947667  | 80.973842  | 6.0126326  | 213.35609  | rice  |  |
| 25 | 83 | 41 | 43 | 21.052536  | 82.678395  | 6.2540285  | 233.10758  | rice  |  |
| 26 | 98 | 47 | 37 | 23.483813  | 81.332651  | 7.3754829  | 224.05812  | rice  |  |
| 27 | 66 | 53 | 41 | 25.075635  | 80.523891  | 7.7789152  | 257.00389  | rice  |  |
| 28 | 97 | 59 | 43 | 26.359272  | 84.044036  | 6.28650017 | 271.358613 | rice  |  |

## Milestone 2: Model Development

### 1.Data Preprocessing

- Performed data preprocessing to clean and organize the dataset for efficient model training.

### 2.Train-Test Split

- Split the dataset into training and testing sets to evaluate the model's performance accurately.

### 3.Machine Learning Algorithm

- Applied the Random Forest Classifier algorithm for crop recommendation prediction.

### 4.Model Training

- Trained the machine learning model using the prepared training dataset.

## 5. Model Saving

- Saved the trained model as crop\_model.pkl using Pickle for future predictions.

## Model Accuracy:

```
Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\VR Digitals\Documents\Smart Crop>python train_model.py
Model Accuracy: 99.32%
Model saved successfully as crop_model.pkl

C:\Users\VR Digitals\Documents\Smart Crop>|
```

## Milestone 3: Flask Backend

### 1. Created Flask Application using app.py

The Flask application was developed using the app.py file to manage the backend functionality.

### 2. Implemented Routes

Flask routes were created to handle user requests and navigate between different application functions.

### 3. Loaded Trained Model

The trained machine learning model was loaded using Pickle for making crop predictions.

### 4. Accepted User Input

The application accepts soil and environmental parameters entered by the user through the web form.

### 5. Predicted Crop

The trained model processes the input values and predicts the most suitable crop.

### 6. Returned Prediction to Webpage

The predicted crop is displayed on the webpage as the final recommendation.

## app.py

```

app.py > ...
You, 4 hours ago | 1 author (You)
1 from flask import Flask, render_template, request
2 import sqlite3
3 import joblib
4 from datetime import datetime
5
6 app = Flask(__name__)
7
8 model = joblib.load("crop_model.pkl")
9
10
11 def create_database():
12     conn = sqlite3.connect("database.db")
13     cursor = conn.cursor()
14
15     cursor.execute("""
16         CREATE TABLE IF NOT EXISTS prediction_history (
17             id INTEGER PRIMARY KEY AUTOINCREMENT,
18             nitrogen REAL,
19             phosphorous REAL,
20             potassium REAL,
21             temperature REAL,
22             humidity REAL,
23             ph REAL,
24             rainfall REAL,
25             recommended_crop TEXT,
26             date TEXT
27         )
28     """)
29
30     conn.commit()
31     conn.close()
32
33
34 create_database()
35
  
```

## Milestone 4: Frontend Development

### 1.Designed Responsive Interface using HTML, CSS, and JavaScript

A responsive and user-friendly interface was designed using HTML, CSS, and JavaScript to provide a smooth user experience.

#### HTML CODE:

```

index.html
You, 3 days ago | 1 author (You)
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>OptiCrop AI Crop Recommendation</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>

<body>
  You, 3 days ago • Initial commit - OptiCrop AI Crop Recommendation...
  <div class="container">
    <h1>OptiCrop AI Crop Recommendation</h1>
    <form action="/predict" method="POST">
      <div class="input-box">
        <label>Nitrogen (N)</label>
        <input type="number" step="any" name="nitrogen" required>
      </div>
    </form>
  </div>
  
```

#### CSS CODE:

```
static > # style.css
You, 5 hours ago | 1 author (You)
You, 3 days ago • Initial commit - OptiCrop AI Crop Recommendation...
1 body{
2   background: linear-gradient(135deg, #E8F5E9 0%, #B2DFDB 100%);
3   font-family:Arial;
4 }
5
6 .container{
7   width:400px;
8   margin:40px auto;
9   background:#4CAF50;
10  padding:25px;
11  border-radius:10px;
12  text-align:center;
13  box-shadow:0 0 10px gray;
14 }
15
16 input{
17   width:90%;
18   padding:10px;
19   margin:8px;
20 }
21
22 button{
23   padding:10px 20px;
24   background:green;
25   color:white;
26   border:none;
27   cursor:pointer;
28 }
29
30 button:hover{
31   background:darkgreen;
32 }
33
```

## 2.Created Input Form

An input form was developed to collect soil and environmental parameters from the user for crop prediction.

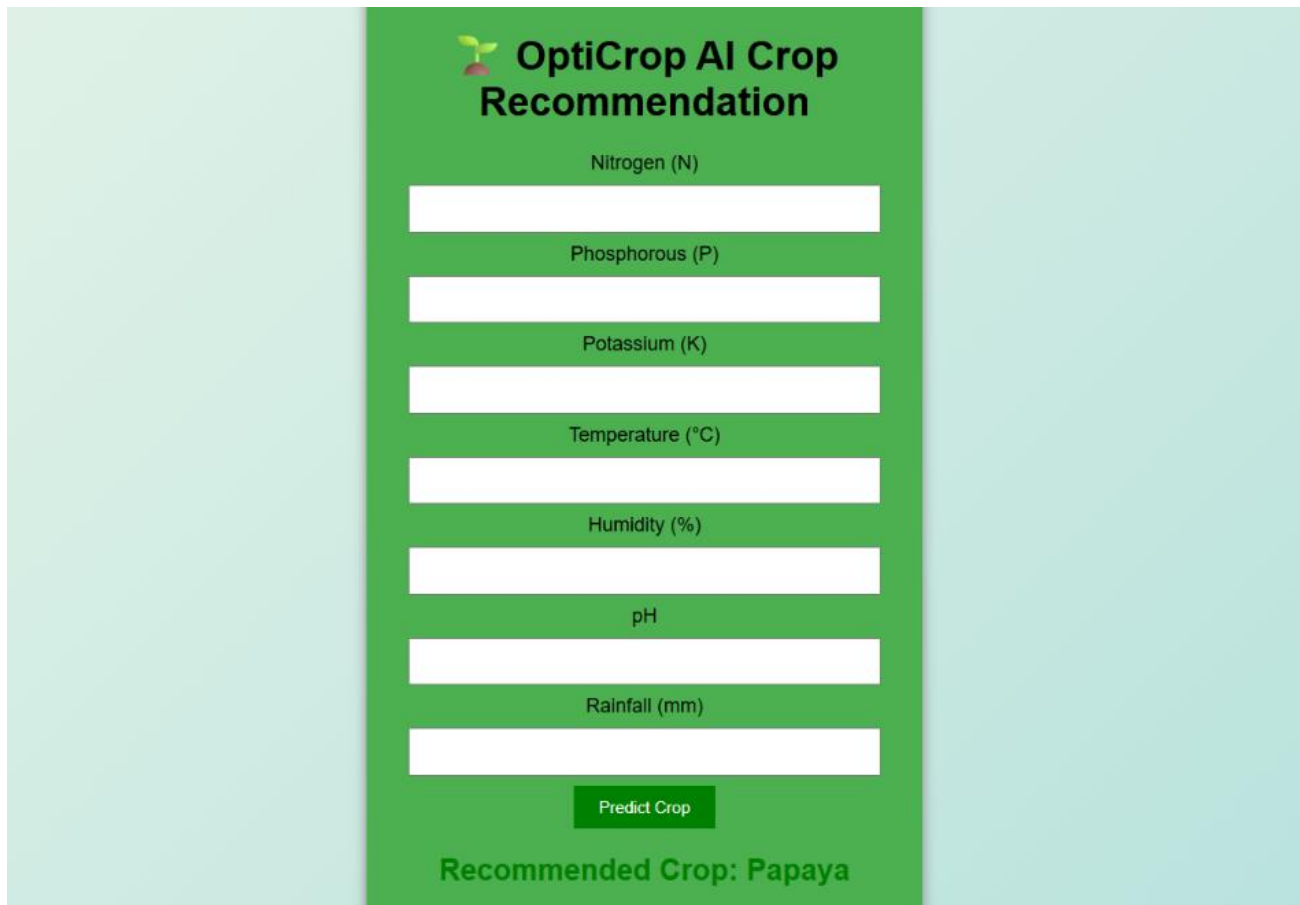
|                      |
|----------------------|
| Nitrogen (N)         |
| <input type="text"/> |
| Phosphorous (P)      |
| <input type="text"/> |
| Potassium (K)        |
| <input type="text"/> |
| Temperature (°C)     |
| <input type="text"/> |
| Humidity (%)         |
| <input type="text"/> |
| pH                   |
| <input type="text"/> |
| Rainfall (mm)        |
| <input type="text"/> |

### 3. Displayed Predicted Crop

The predicted crop recommendation is displayed on the webpage based on the values entered by the user.

After giving some input it predicted as Papaya.

### Crop Recommended:



The screenshot shows a web application titled "OptiCrop AI Crop Recommendation" with a green background. It features a series of input fields for various parameters: Nitrogen (N), Phosphorous (P), Potassium (K), Temperature (°C), Humidity (%), pH, and Rainfall (mm). Below these fields is a green button labeled "Predict Crop". At the bottom, the text "Recommended Crop: Papaya" is displayed in green.

## Milestone 5 : Testing

### 1. Tested Application Using Various Soil Values

The application was tested with different combinations of soil and environmental parameters to ensure proper functionality.

### 2. Verified Predictions

The predicted crop recommendations were verified against the expected results to ensure reliable performance.



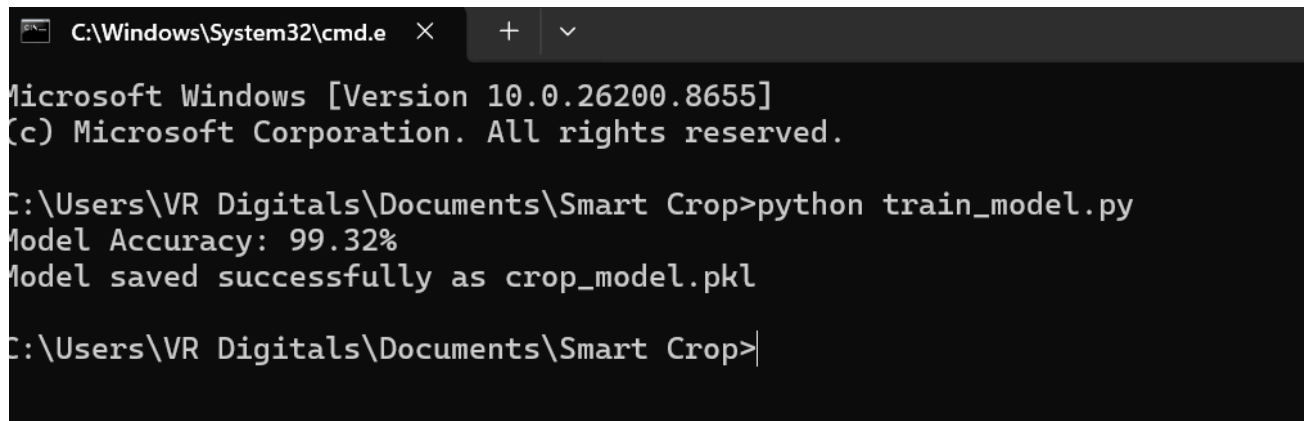
### 3.Checked Invalid Inputs

The application was tested with invalid and incomplete inputs to verify proper input validation and error handling.

### 4.Verified Output Accuracy

The output generated by the machine learning model was evaluated to ensure accurate and consistent crop recommendations.

### Accuracy:



```
C:\Windows\System32\cmd.e  X  +  v

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

C:\Users\VR Digitals\Documents\Smart Crop>python train_model.py
Model Accuracy: 99.32%
Model saved successfully as crop_model.pkl

C:\Users\VR Digitals\Documents\Smart Crop>
```

## GitHub

### 1.Uploaded Project to GitHub

The completed project was uploaded to GitHub for version control and online access.

### 2.Created Repository

A new GitHub repository was created to store and manage all the project files.

### 3.Committed Project Files

All project files were committed with appropriate commit messages to track the development progress.

### 4.Managed Versions Using Git

Git was used to manage different versions of the project, making it easier to update and maintain the code.

### GitHub Repository:

main

1 Branch

0 Tags

Go to file

T

Add file

Code

Premalikhitha

Add runtime.txt with required package versions

a48dbfa · 18 hours ago

5 Commits

|                                               |                                                        |              |
|-----------------------------------------------|--------------------------------------------------------|--------------|
| <div><div></div>static</div>                  | Add files via upload                                   | 4 days ago   |
| <div><div></div>templates</div>               | Add files via upload                                   | 4 days ago   |
| <div><div></div>app.py</div>                  | Update app.py to run on host 0.0.0.0 and port 5000     | 19 hours ago |
| <div><div></div>crop_model.pkl</div>          | Add files via upload                                   | 4 days ago   |
| <div><div></div>crop_recommendation.csv</div> | Add files via upload                                   | 4 days ago   |
| <div><div></div>database.db</div>             | Add files via upload                                   | 4 days ago   |
| <div><div></div>database.txt</div>            | Add files via upload                                   | 4 days ago   |
| <div><div></div>requirements.txt</div>        | Add initial requirements for Flask and data processing | 19 hours ago |
| <div><div></div>runtime.txt</div>             | Add runtime.txt with required package versions         | 18 hours ago |
| <div><div></div>train_model.py</div>          | Add files via upload                                   | 4 days ago   |

# Deployment

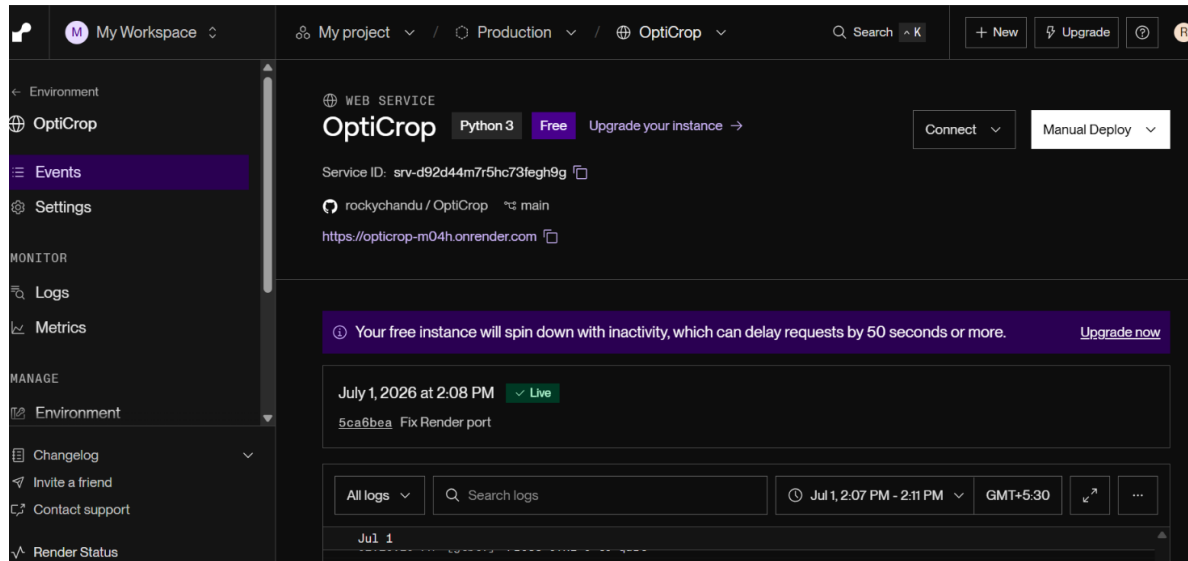
## 1.Hosted Application on Render

The application was hosted on the Render cloud platform, making it accessible through a public web URL.

## 2.Connected GitHub Repository

The GitHub repository was connected to Render to enable automatic deployment from the latest project code.

## Opening OptiCrop Repository in Render:



### 3.Configured Build Command

The build command was configured to install all the required project dependencies during deployment.

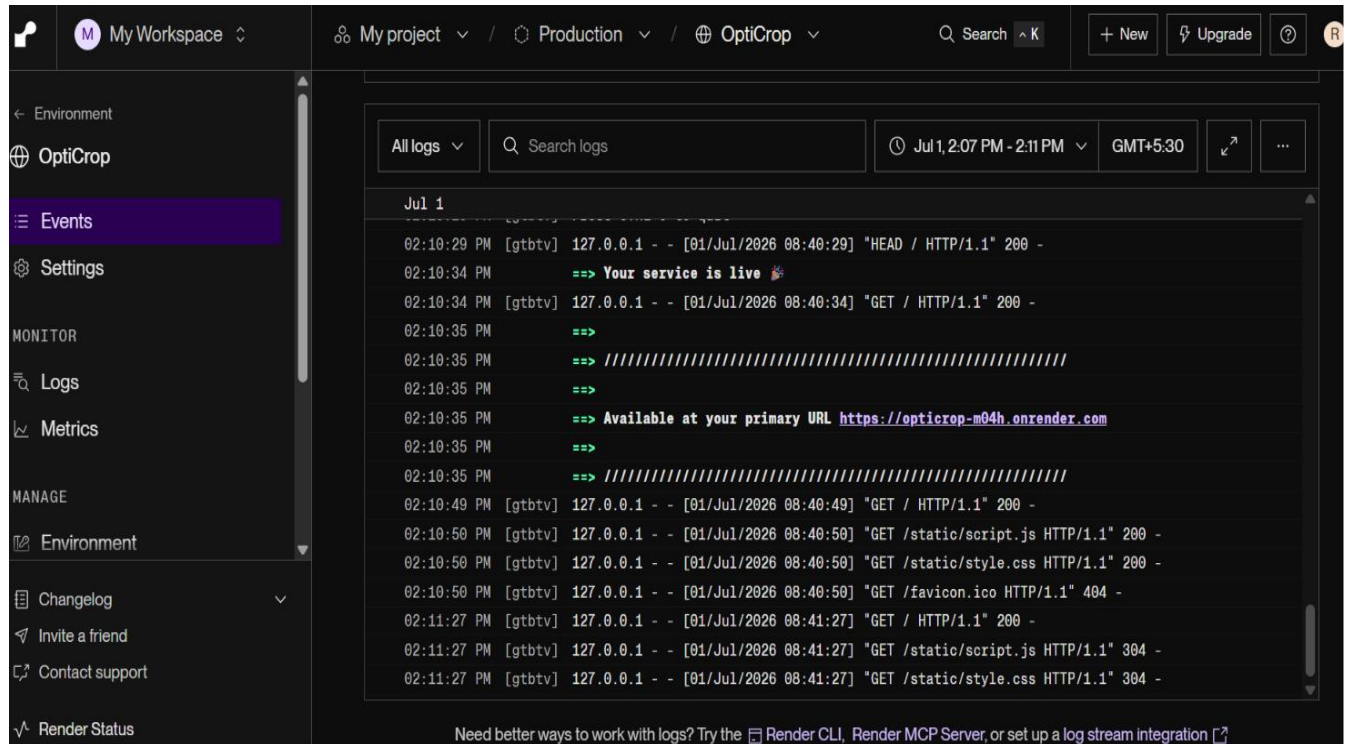
### 4.Configured Start Command

The start command was set to launch the Flask application and make it available online.

### 5.Successfully Deployed Application

The application was successfully deployed on Render and tested to ensure it was accessible and functioning correctly.

**After Deploying,Link generated to access:**



The screenshot displays the OptiCrop application interface. The left sidebar contains navigation options: Environment, OptiCrop, Events (highlighted), Settings, MONITOR, Logs, Metrics, MANAGE, Environment, Changelog, Invite a friend, Contact support, and Render Status. The main area shows the logs for the 'OptiCrop' project. The logs are filtered for 'All logs' and show a search bar and a time range of 'Jul 1, 2:07 PM - 2:11 PM' in 'GMT+5:30'. The log entries are as follows:

```

Jul 1
02:10:29 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:40:29] "HEAD / HTTP/1.1" 200 -
02:10:34 PM ==> Your service is live 🎉
02:10:34 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:40:34] "GET / HTTP/1.1" 200 -
02:10:35 PM ==>
02:10:35 PM ==> //////////////////////////////////////
02:10:35 PM ==> Available at your primary URL https://opticrop-m04h.onrender.com
02:10:35 PM ==>
02:10:35 PM ==> //////////////////////////////////////
02:10:49 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:40:49] "GET / HTTP/1.1" 200 -
02:10:50 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:40:50] "GET /static/script.js HTTP/1.1" 200 -
02:10:50 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:40:50] "GET /static/style.css HTTP/1.1" 200 -
02:10:50 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:40:50] "GET /favicon.ico HTTP/1.1" 404 -
02:11:27 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:41:27] "GET / HTTP/1.1" 200 -
02:11:27 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:41:27] "GET /static/script.js HTTP/1.1" 304 -
02:11:27 PM [gtbtv] 127.0.0.1 - - [01/Jul/2026 08:41:27] "GET /static/style.css HTTP/1.1" 304 -
  
```

At the bottom, a message suggests: "Need better ways to work with logs? Try the [Render CLI](#), [Render MCP Server](#), or set up a [log stream integration](#)."

Deployed Link : <https://opticrop-m04h.onrender.com>

## Prediction Page

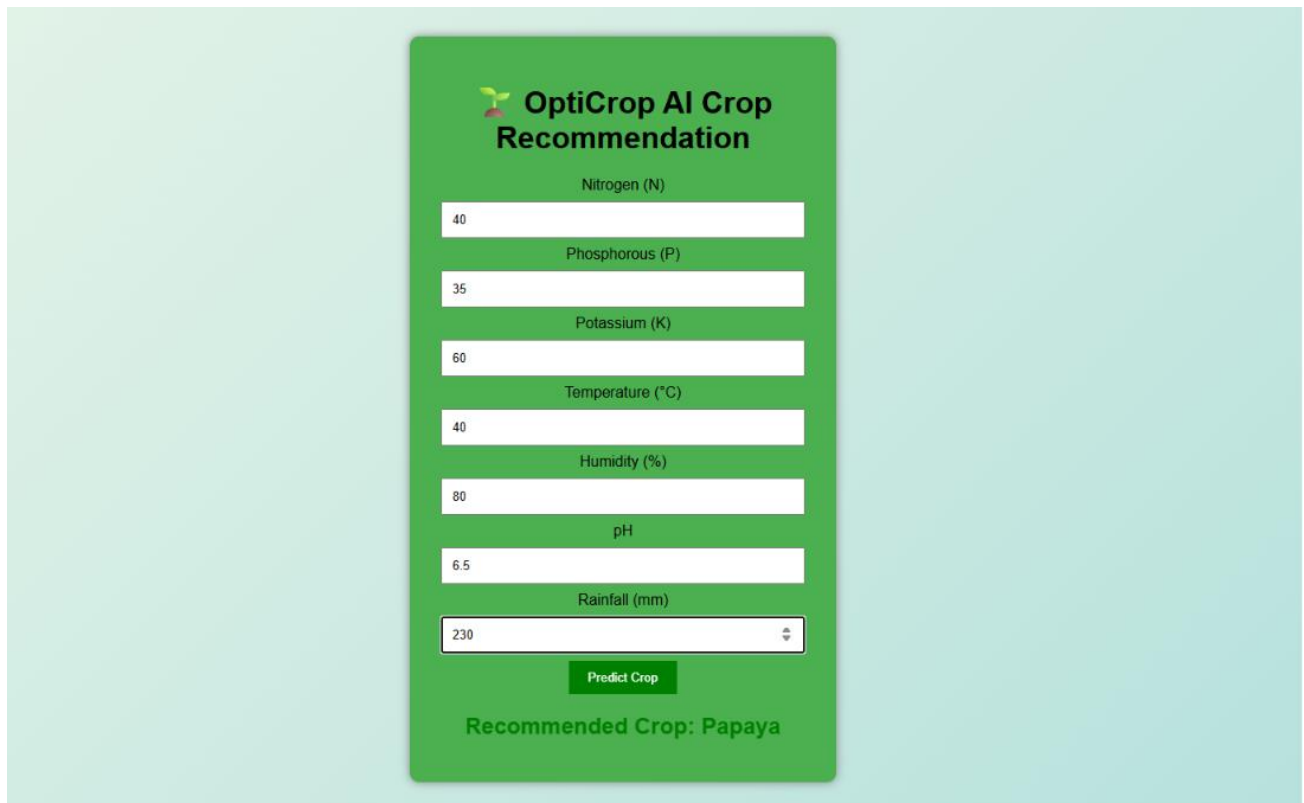
### 1. Machine Learning Model Predicts Suitable Crop

The trained machine learning model analyzes the user-provided soil and environmental parameters to predict the most suitable crop.

### 2. Displays Recommendation

The predicted crop recommendation is displayed clearly on the webpage for the user.

## Prediction of a Crop



The screenshot shows a web application titled "OptiCrop AI Crop Recommendation" with a green background. It features a form with input fields for various parameters: Nitrogen (N) with value 40, Phosphorous (P) with value 35, Potassium (K) with value 60, Temperature (°C) with value 40, Humidity (%) with value 80, pH with value 6.5, and Rainfall (mm) with value 230. A green "Predict Crop" button is located below the form. At the bottom, it displays the "Recommended Crop: Papaya".

| Parameter        | Value |
|------------------|-------|
| Nitrogen (N)     | 40    |
| Phosphorous (P)  | 35    |
| Potassium (K)    | 60    |
| Temperature (°C) | 40    |
| Humidity (%)     | 80    |
| pH               | 6.5   |
| Rainfall (mm)    | 230   |

**Predict Crop**

**Recommended Crop: Papaya**

## **Technologies Used:**

| Technology   | Purpose              |
|--------------|----------------------|
| Python       | Backend              |
| Flask        | Web Framework        |
| HTML         | Frontend             |
| CSS          | Styling              |
| JavaScript   | Interactivity        |
| Scikit-learn | Machine Learning     |
| Pandas       | Data Processing      |
| NumPy        | Numerical Operations |
| Pickle       | Model Saving         |
| GitHub       | Version Control      |
| Render       | Deployment           |

## **Advantages:**

### ✧ **Fast Prediction**

The system quickly predicts the most suitable crop based on the user-provided input values.

### ✧ **User-Friendly**

The application provides a simple and intuitive interface that is easy for users to navigate.

### ✧ **AI-Based Recommendation**

The system uses a trained machine learning model to generate intelligent crop recommendations.

### ✧ **Accurate Results**

The trained model provides reliable and accurate crop predictions using soil and environmental data.

### ✧ **Supports Smart Farming**

The application helps farmers make informed decisions, promoting efficient and sustainable agricultural practices.

### ✧ **Easy to Use**

Users can easily enter the required parameters and receive crop recommendations with just a few clicks.

## **Applications:**

### ✧ **Smart Agriculture**

The system supports smart agriculture by using machine learning to improve crop selection and farming decisions.

### ✧ **Precision Farming**

It helps farmers choose suitable crops based on soil and environmental conditions, enabling precision farming.

### ✧ **Farmer Decision Support**

The application assists farmers in selecting the most appropriate crop to maximize productivity.

### ✧ **Agricultural Research**

The system can be used by researchers to analyze crop recommendations based on different agricultural parameters.

### ✧ **Crop Planning**

The application helps in planning crop cultivation by recommending suitable crops for specific soil and climate conditions.

## **Future Scope:**

### Future Enhancements

- Fertilizer Recommendation
- Disease Detection
- Weather API Integration
- Mobile Application
- Multi-language Support
- User Login System
- Cloud Database

## **Conclusion:**

The OptiCrop – AI Crop Recommendation System is a machine learning-based web application developed to assist users in selecting the most suitable crop based on soil nutrients and environmental conditions. By taking inputs such as Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, and Rainfall, the system predicts the most appropriate crop using a trained machine learning model. This helps users make informed decisions and promotes efficient agricultural practices.

The project was developed using Python, Flask, HTML, CSS, JavaScript, Pandas, NumPy, and Scikit-learn. A responsive and user-friendly interface was designed to provide a smooth user experience, while the backend efficiently processes user inputs and generates accurate crop recommendations. The trained model was integrated into the Flask application using Pickle, and the system was thoroughly tested with different input values to ensure reliable performance and accurate predictions.

Finally, the project was uploaded to GitHub for version control and successfully deployed on Render, making it accessible through a public web URL. Overall, the OptiCrop system demonstrates the practical application of Artificial Intelligence in agriculture and highlights how machine learning can support smart farming by improving crop selection and increasing productivity. In the future, the project can be enhanced by adding fertilizer recommendations, weather forecasting, crop disease detection, multilingual support, and user authentication to provide a more comprehensive smart agriculture solution.